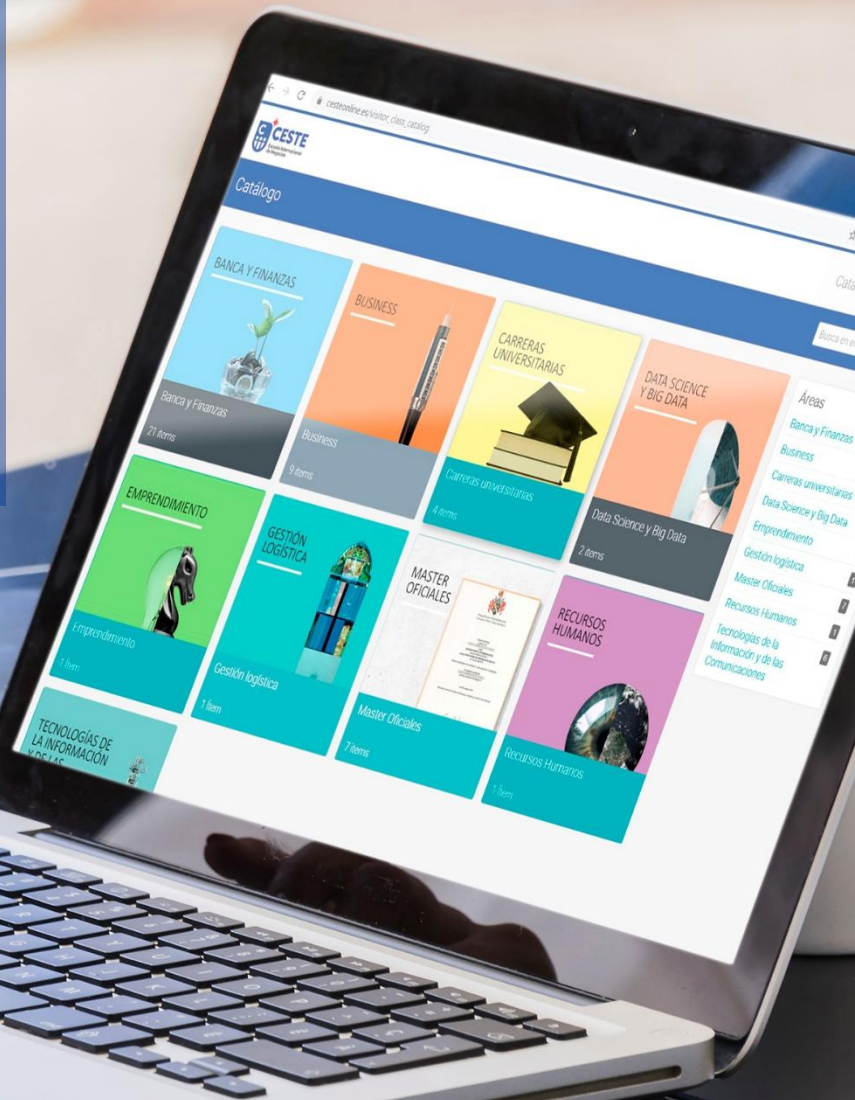


CESTE

Escuela Internacional de Negocios

Zaragoza (España)



Spark Structured Streaming INTRO



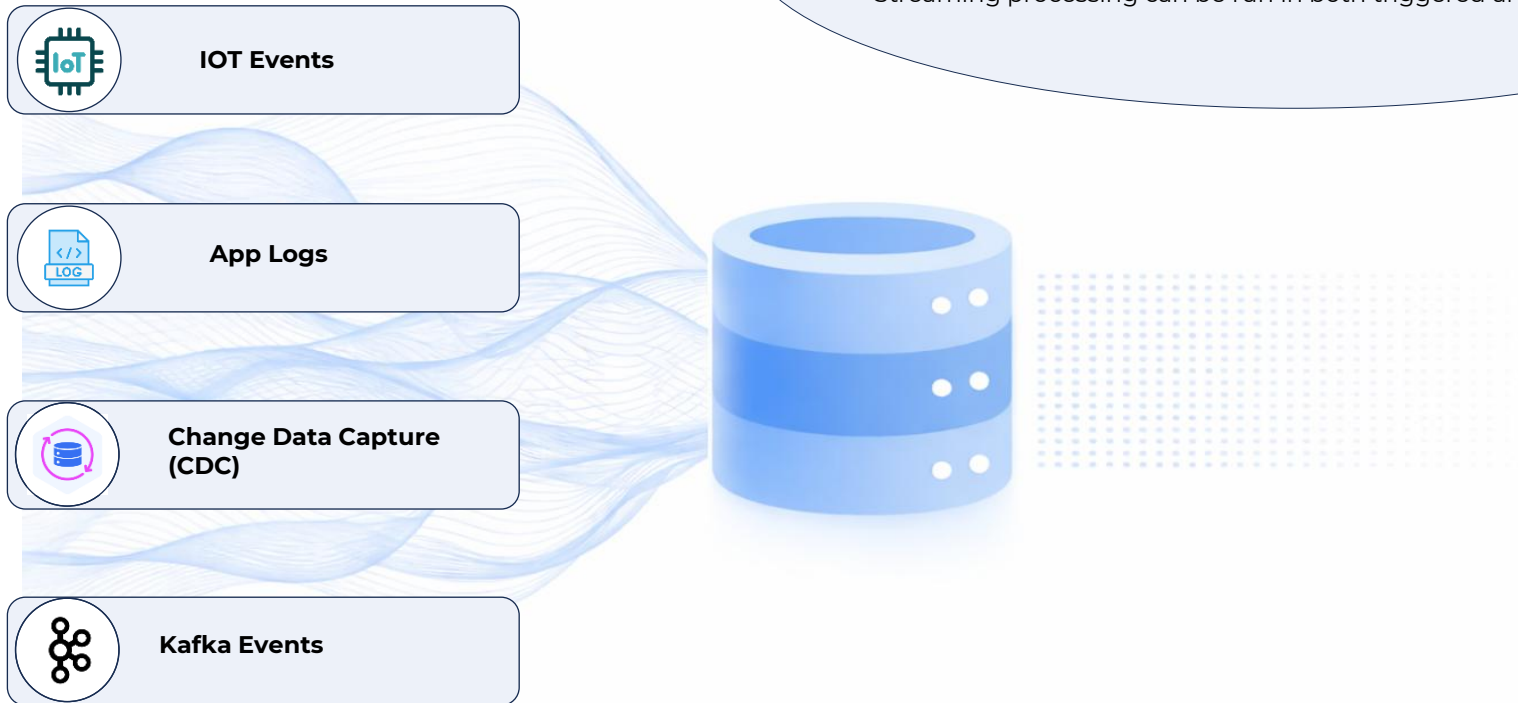
SPARK STREAMING INTRO

1. Que es Streaming
2. Spark Streaming – Example
3. DEMO I

Streaming

Streaming meaning:

- Processes only new data iteratively from the source
- Cloud Object storage and Delta Lake can be streaming sources!
- Streaming processing can be run in both triggered and continuous manners!



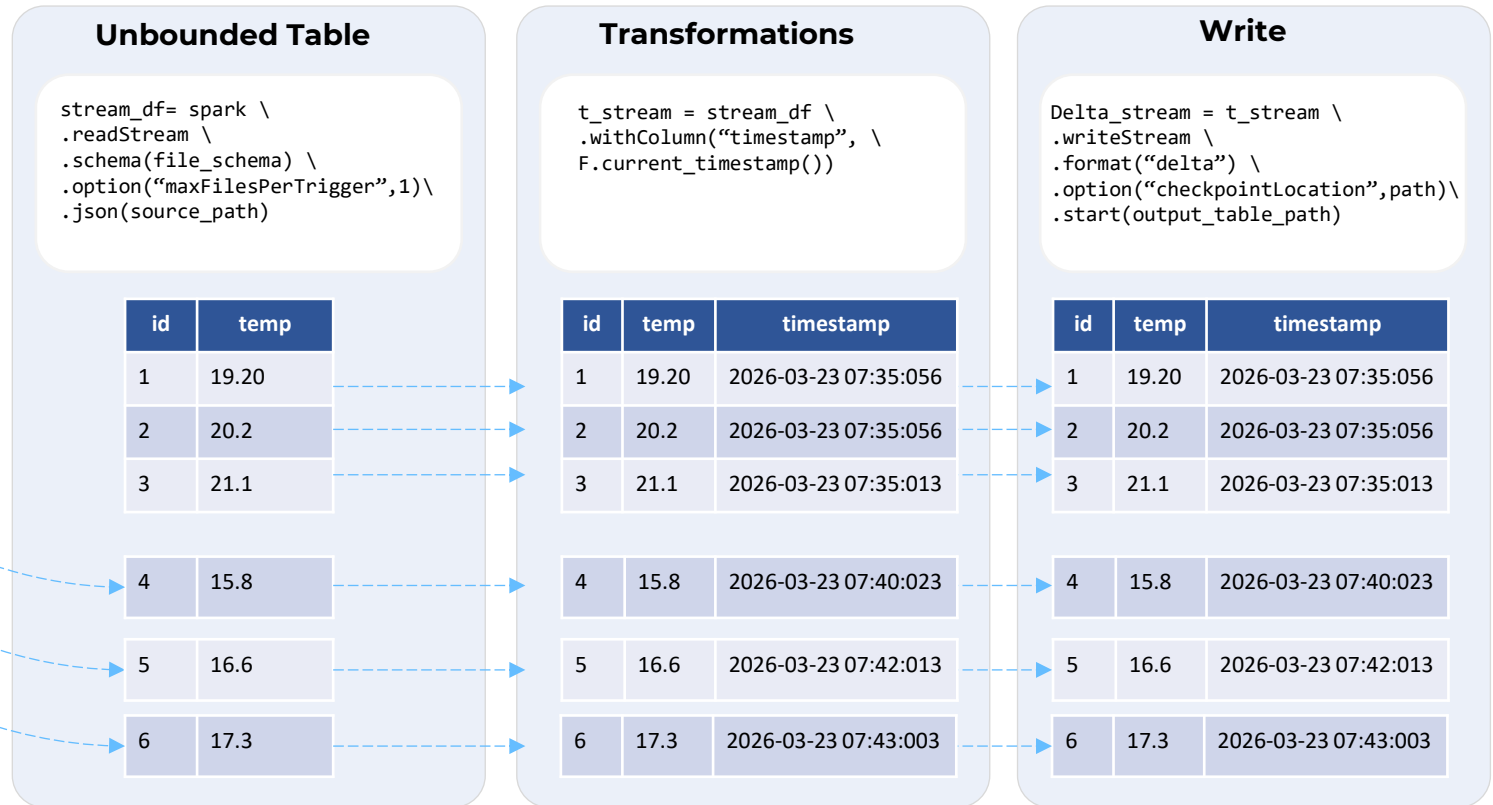
Sensors, IoT devices, social networks, and online transactions all generate data that needs to be monitored constantly and acted upon quickly. As a result, the need for large-scale, real-time stream processing is more evident than ever before.

SPARK STRUCTURED STREAMING

*how many Spark Actions?



Data Stream



In Structured Streaming, **a data stream is treated as a table that is being continuously appended**. This leads to a stream processing model that is very similar to a batch processing model. You express your streaming computation as a standard batch-like query as on a static table, but Spark runs it as an incremental query on the unbounded input table.

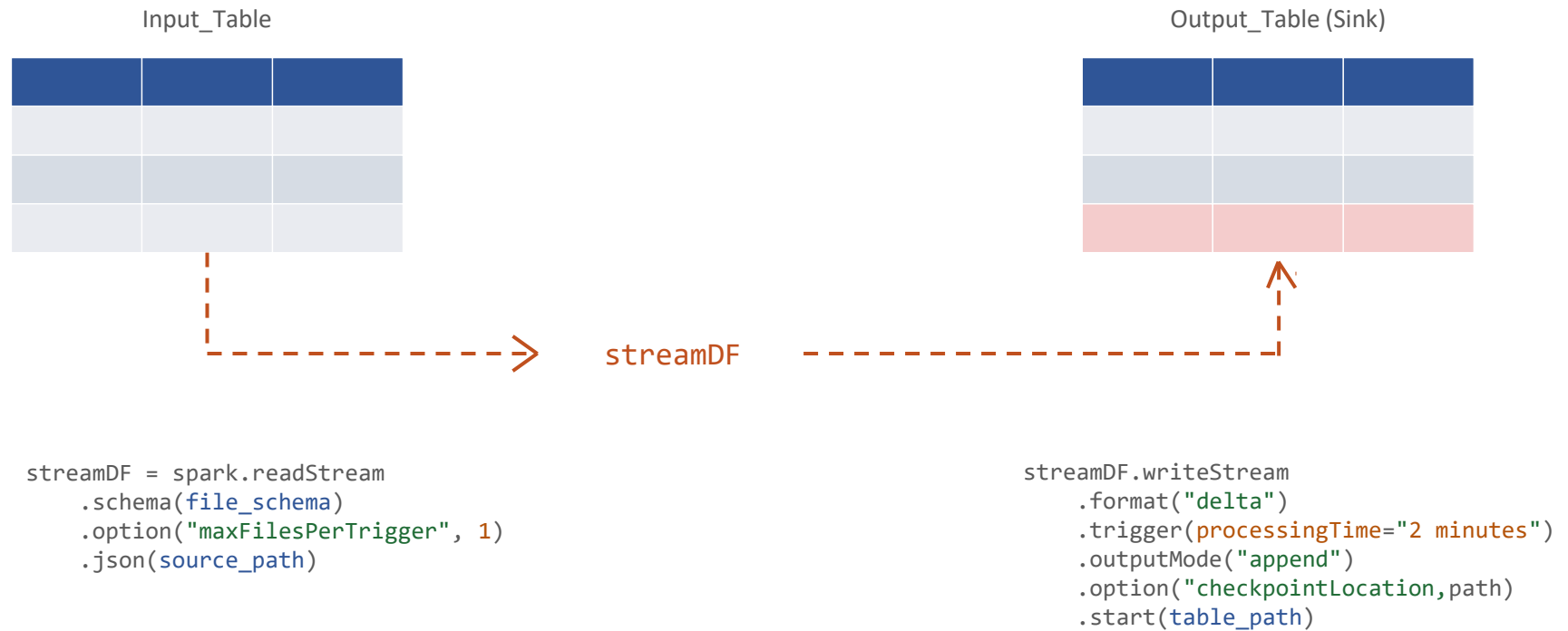
DEMO 3.1

Structured Streaming

SPARK STREAMING INTRO

1. ReadStream
 1. Schema declaration
 2. Options
 3. Source (input)
2. WriteStream
 1. Format
 2. Trgger
 3. Outputmode
 4. Checkpoint

Input Streaming Table

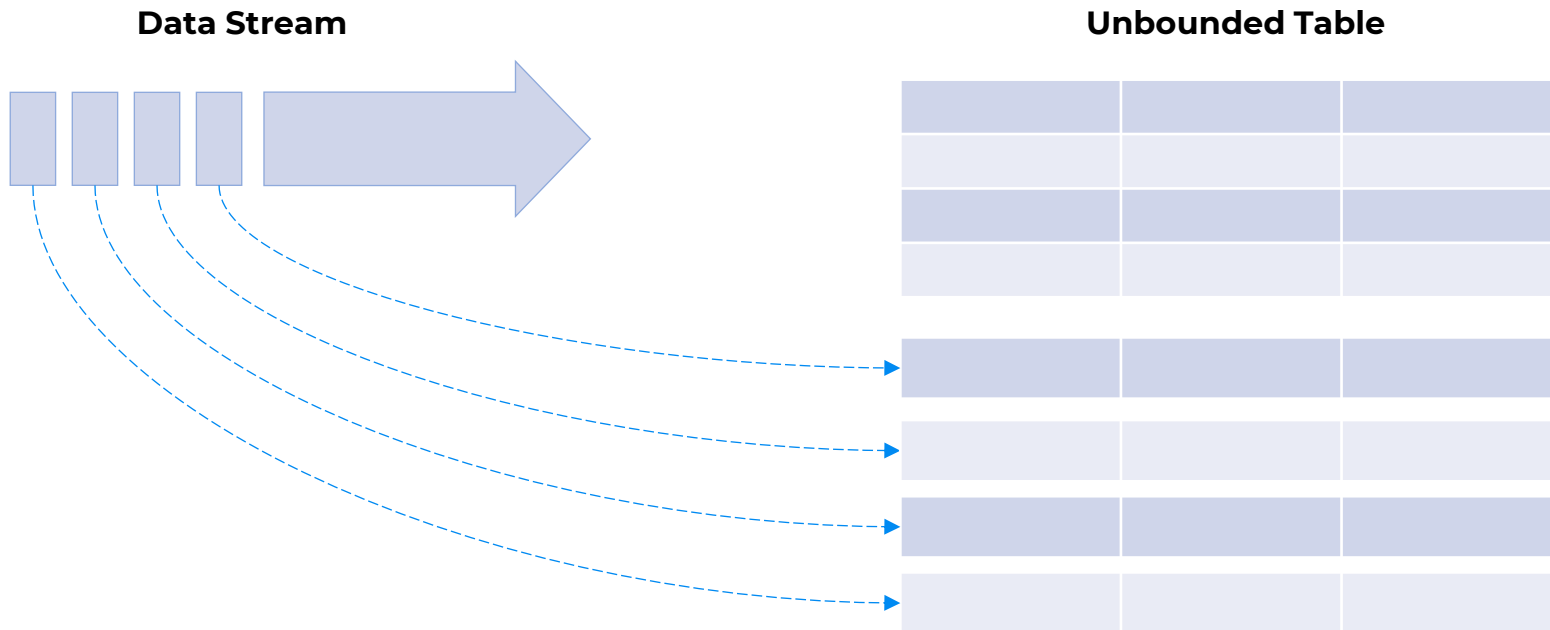


Read Stream - Deatil

```
streamDF = spark.readStream  
    .schema(file_schema)  
    .option("maxFilesPerTrigger", 1)  
    .json(source_path)
```

SPARK STRUCTURED STREAMING

Consider the input data stream as the input table. Every data item that is arriving on the stream is like a new row being appended to the input table.



In Structured Streaming, **a data stream is treated as a table that is being continuously appended**. This leads to a stream processing model that is very similar to a batch processing model. You express your streaming computation as a standard batch-like query as on a static table, but Spark runs it as an incremental query on the unbounded input table.

readStream — .schema()

```
streamDF = spark.readStream
  .schema(file_schema)
  .option("maxFilesPerTrigger", 1)
  .json(...) / .parquet(...) / ...
```

¿Cuándo es obligatorio y cuándo no?

Formato	Schema	Motivo
.json(path)	Obligatorio	Texto plano sin schema embebido
.csv(path)	Obligatorio	Texto plano sin schema embebido
.parquet(path)	No obligatorio	El schema está dentro del fichero Parquet
.format("delta").load()	No obligatorio	Delta guarda el schema en _delta_log
Auto Loader	No obligatorio	Lo infiere y persiste en cloudFiles.schemaLocation

¿Por qué es obligatorio en streaming?

En batch, Spark escanea todos los ficheros antes de empezar e infiere el schema. En streaming no puede — el stream no sabe cuántos ficheros llegarán ni cuándo, así que no hay escaneo previo posible. El schema debe declararse antes de arrancar.

readStream — .option("maxFilesPerTrigger")

```
streamDF = spark.readStream
  .schema(file_schema)
  .option("maxFilesPerTrigger", 1)
  .json(source_path)
```

¿Qué hace?

Limita cuántos ficheros nuevos se leen por microbatch. Spark detecta todos los ficheros pendientes en la fuente pero solo procesa el número indicado por trigger, guardando el resto para el siguiente.

Impacto según el valor elegido

Valor	Latencia	Throughput	Cuándo usarlo
1	Alta	Bajo	Demos y desarrollo — ver los batches uno a uno
10 - 100	Media	Medio	Producción con carga moderada — balance coste/latencia
Sin opción	Baja	Alto	Backfill rápido — procesa todo el backlog de golpe

El trade-off clave: latencia vs coste de memoria

Valor bajo → batches pequeños → menos memoria por batch, más overhead de commit Delta y checkpoint por fichero procesado.

Valor alto → batches grandes → más memoria necesaria, pero menos overhead fijo por fichero. Con ficheros muy grandes puede provocar OOM.

Alternativa: `.option("maxBytesPerTrigger", "128m")` — limita por tamaño en bytes en vez de por número de ficheros

Write Stream - Deatil

```
streamDF.writeStream
  .format("delta")
  .trigger(processingTime="2 minutes")
  .outputMode("append")
  .option("checkpointLocation",path)
  .start(table_path)
```

Trigger Intervals

```
streamDF.writeStream  
  .trigger(processingTime="2 minutes")  
  .outputMode("append")  
  .option("checkpointLocation", checkpoint_path)  
  .start(table_path)
```

Trigger	Method call	Behavior
Unspecified		Default: processingTime="500ms" Microbatches continuos, nunca para
Fixed interval	.trigger(processingTime ="5 minutes")	Microbatch cada X tiempo Nunca para
Triggered batch	.trigger(once =True)	Todo en un único batch Ignora maxFilesPerTrigger, para al terminar
Triggered micro-batches	.trigger(availableNow =True)	Múltiples microbatches hasta vaciar Respetar maxFilesPerTrigger, para al terminar ✓

Output Modes

```
streamDF.writeStream  
  .trigger(availableNow=True)  
  .outputMode("append")  
  .option("checkpointLocation", checkpoint_path)  
  .start(table_path)
```

Mode	Method call	Behavior
Append (Default)	<code>.outputMode("append")</code>	Solo las filas nuevas del microbatch se añaden a la tabla. No modifica filas existentes. Único modo compatible con Delta sin agregaciones.
Complete	<code>.outputMode("complete")</code>	La tabla entera se sobrescribe en cada batch. Necesario para agregaciones globales (ej: COUNT total). Muy costoso en volumen alto.
Update	<code>.outputMode("update")</code>	Solo las filas que han cambiado se escriben. Para agregaciones con estado (ej: COUNT por ventana). No soportado por todos los sinks.

Checkpointing

```
streamDF.writeStream  
  .trigger(availableNow=True)  
  .outputMode("append")  
  .option("checkpointLocation", checkpoint_path)  
  .start(table_path)
```

1

Guarda el estado del stream

Persiste qué batches se han completado, hasta qué offset se ha leído y qué ficheros ya se procesaron.

2

Permite recuperación ante fallos

Si el stream falla o se reinicia, retoma exactamente desde donde lo dejó. Garantía de exactly-once.

3

No se puede compartir entre streams

Cada stream necesita su propio checkpoint_path único. Compartirlo entre dos streams corrompe el estado.

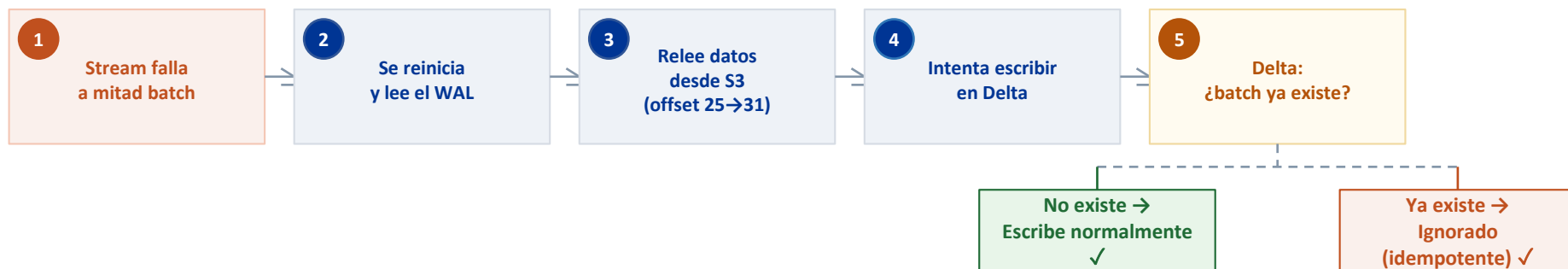
4

Bórralo si cambias el schema o la fuente

Si modificas readStream (schema, source_path...) el checkpoint antiguo es incompatible — hay que borrarlo.

Garantees — cómo funciona exactamente

¿Qué pasa cuando el stream falla a mitad de un batch?



✓ Resultado: exactly-once garantizado bajo cualquier condición de fallo — ni duplicados ni pérdidas

Write-ahead log (WAL)

Spark escribe ANTES de ejecutar el batch qué rango de offsets va a procesar. Si cae a mitad, al reiniciarse lee ese log y sabe exactamente qué tenía pendiente.

Fuente repetible

La fuente debe poder releer datos desde un offset anterior. S3 y Kafka lo garantizan — los datos siguen ahí. Una API REST en tiempo real no sería repetible.

Sink idempotente (Delta)

Delta identifica cada batch por su número en el `_delta_log`. Si intentas escribir el mismo batch dos veces, lo detecta y lo ignora. Mismo input → mismo resultado siempre.

Unsupported Operations

La mayoría de operaciones en un streaming DataFrame son idénticas a un DataFrame estático. Pero hay excepciones.

! Operaciones no soportadas en streaming DataFrames

Sorting / ORDER BY

Para ordenar necesitas TODOS los datos. La tabla en streaming es unbounded — nunca tienes todos los datos, siempre puede llegar un valor nuevo que cambie el orden. Es lógicamente imposible.

Deduplication global

Sin watermark, Spark tendría que guardar en memoria todos los IDs vistos desde el inicio del stream — memoria ilimitada que crece indefinidamente hasta colapsar el cluster.

2 Advanced methods — cómo resolver esas limitaciones

Windowing

Acota el stream en ventanas de tiempo finitas

En vez de operar sobre una tabla sin límite, Windowing divide el stream en grupos de tiempo — por ejemplo, "todos los eventos de los últimos 5 minutos". Dentro de esa ventana el conjunto de datos sí es finito, por lo que puedes ordenar, agregar o calcular medias. Una vez la ventana se cierra, los resultados se emiten y el estado se libera.

Watermarking

Gestiona datos tardíos y habilita deduplicación

Define cuánto tiempo está dispuesto a esperar Spark por datos que llegan fuera de orden antes de cerrar una ventana. Por ejemplo, watermark de 10 minutos significa: acepto datos con hasta 10 minutos de retraso, pero después de ese umbral descarto cualquier dato más antiguo y libero el estado. Sin watermark, Spark acumularía estado indefinidamente.

DEMO 3.2

Structured Streaming II

.json() vs Auto Loader — detección de ficheros nuevos

El motor es idéntico — la diferencia es PULL (Spark pregunta) vs PUSH (S3 avisa)

.json(path) — modelo PULL

vs

Auto Loader — modelo PUSH

- 1 LIST completo del directorio S3
- 2 Lee checkpoint (ficheros ya procesados)
- 3 **Anti-join conceptual:
todos_ficheros – ya_procesados**
- 4 Procesa ficheros nuevos

El problema:

El LIST crece con el nº de ficheros acumulados en S3. Con millones de ficheros históricos, cada trigger es costoso aunque solo haya llegado 1 fichero nuevo.

Coste del LIST con el tiempo:



- 1 Fichero nuevo llega a S3
→ evento en cola SQS

- 2 Spark consume la cola
(sin hacer ningún LIST)

- 3 Procesa directamente
los ficheros de la cola

La ventaja:

Sin LIST. Sin anti-join. S3 avisa a Spark directamente. El coste es constante independientemente de cuántos ficheros históricos haya acumulados.

Coste con el tiempo:

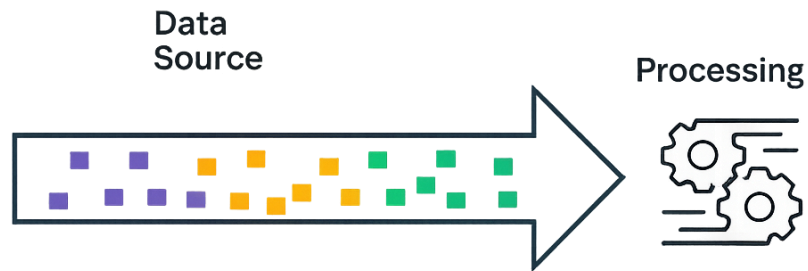


SPARK STREAMING

INTRO

1. Streaming vs Batch
2. Materialized views (Enzyme)
3. When to use what

Streaming Semantics



- Processes only new data iteratively from the source
- Cloud Object storage and Delta Lake can be streaming sources!
- Streaming processing can be run in both triggered and continuous manners!

Why Streaming



Efficient Incremental processing, only new data is being processed

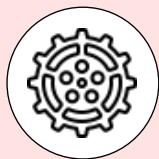


Low Latency unlock use cases demanding lower latency



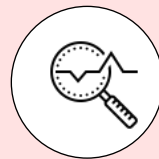
Fault Tolerant recover from where you left off during failures

Why NOT Streaming



Development Complexities:

- Out of order
- Stream – stream JObs



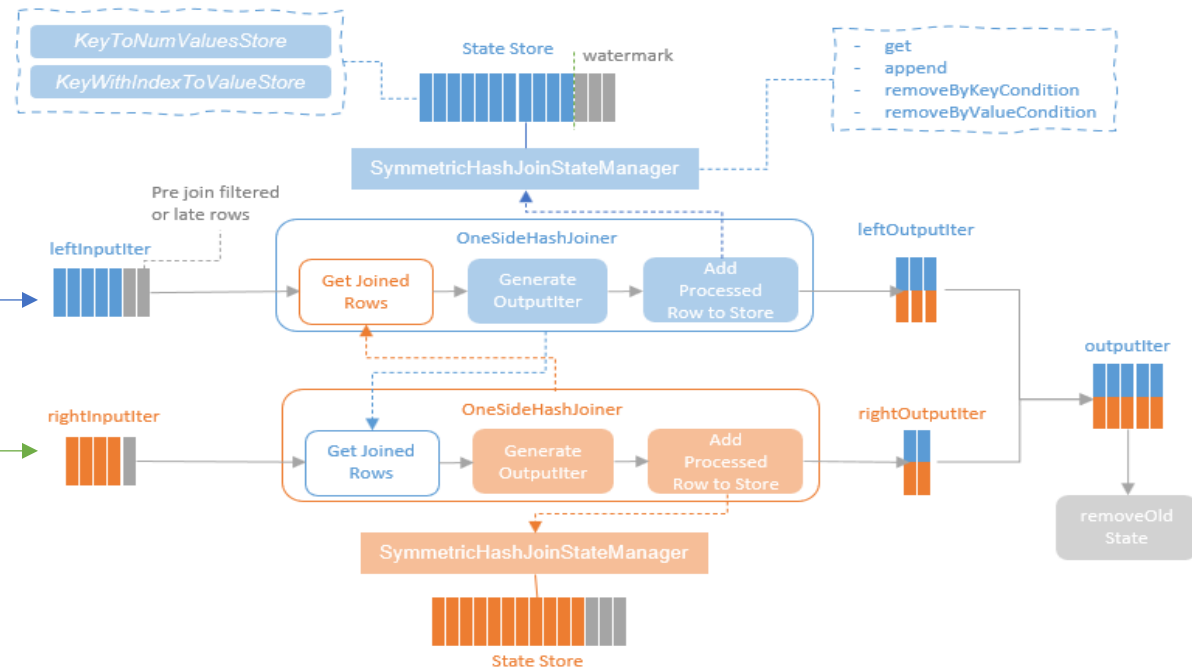
Operational Complexities:

- Large State
- Variant data volumes
- Replay & Rewind

Stream-stream join

ID	Tem	Id_M	time
1	28	M_1	.. 08:42:31
2	29	M_2	.. 08:42:31
3	25	M_2	.. 08:42:31

Id_M	sta	time
M_1	ON	.. 09:42:31
M_2	OFF	.. 08:45:31
M_2	ON	.. 08:4:31



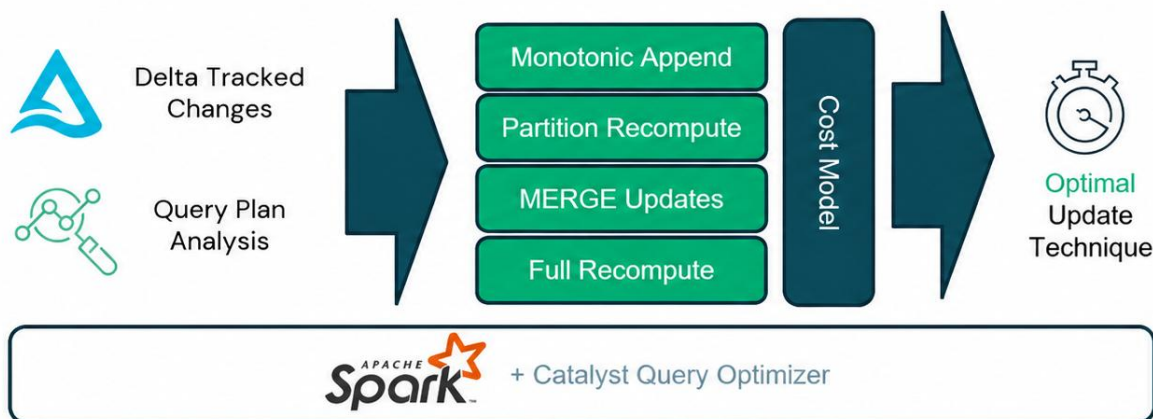
Unlike batch joins, streaming joins require maintaining state on both sides — every row must be stored until a matching row arrives from the other stream.

Streaming vs Batch

	STREAMING	BATCH
PROs	• Efficient, only new data is processed. • Faster, could handle latency requirements from hours to minutes, seconds, and milliseconds.	• Processing logic is simple. • Results are always accurate and reflect all the available data in the source.
CONS	• Processing logic could be complex, especially for stateful processing such as joins, aggregations, deduplications, etc. • Results can not always be accurate, considering out-of-order and late arrival data.	• It is not as efficient; data will be reprocessed at each run. • Slower, could handle latency requirements from hours to minutes, but not seconds or milliseconds.

Materialized views (Enzyme)

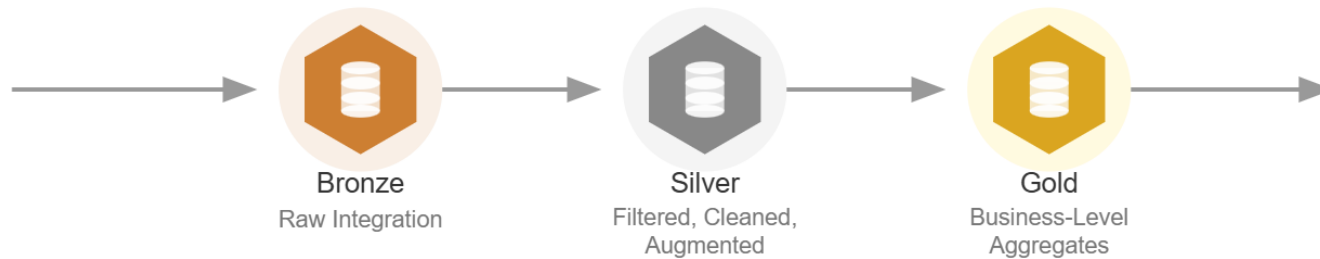
- Incrementalization engine for batch semantics – only process changes from source tables versus full refresh, whenever possible
- Available in Materialized Views and Serverless



Enzyme evalúa las 4 estrategias y elige la óptima según el coste estimado, usando el **Catalyst Query Optimizer** de Spark como base para los planes de ejecución.

When to use what?

Raw Data



Bronze

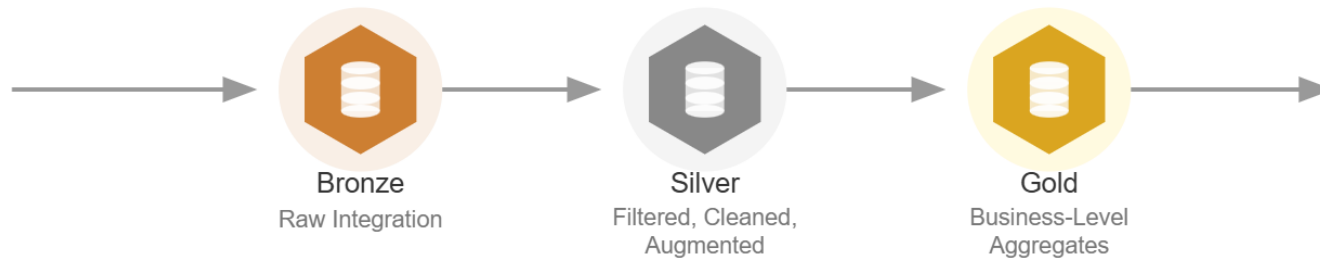
- Ingestion workloads.
- Typically involves no or stateless processing for incremental append from data sources.
- The size of data is typically larger.

Recommendation

- Streaming processing is generally a better choice, given users can benefit from the advantages of streaming but not get exposed to the complexities of stateful streaming processing.

When to use what?

Raw Data



Silver

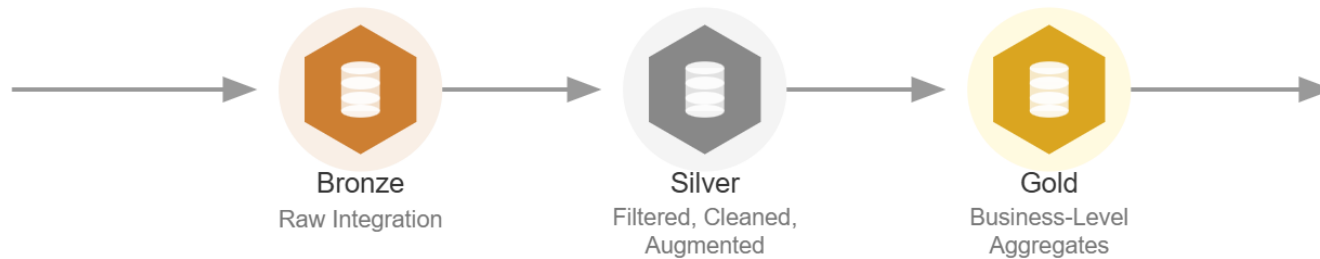
- Transformation workloads.
- Typically involves both stateless processing such as filtering and stateful processing such as joins, aggregations, and deduplications.

Recommendation

- Use batch processing (with incremental refresh in Materialized Views) to avoid the complexities of stateful streaming processing.

When to use what?

Raw Data



Gold

- Last-mile aggregation workloads.
- Typically involves stateful processing such as joins and aggregations.
- The size of data is generally smaller.

Recommendation

- Use batch processing (with incremental refresh in Materialized Views) to avoid the complexities of stateful streaming processing.

Two ways to achieve incremental processing

Structured Streaming



(Sporty, manual transmission)

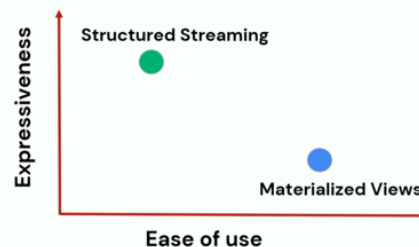
For those who enjoy **control** and **driving**

Materialized Views



(Self-driving car)

For those just care about the **destination**



DEMO 3.3

Structured Streaming III

Lazy Evaluation: Streaming vs. Materialized Views

Una definición declarativa + una acción dispara toda la cadena

⚙️ Spark Streaming

`readStream()`

Lee desde la fuente de datos

LAZY

`.filter() / .select() / ...`

Transformaciones intermedias

LAZY

`writeStream.start()`

Dispara readStream + transforms + escritura continua

⚡ ACTION

💎 Materialized View

`CREATE MATERIALIZED VIEW`

Define la query SELECT, sin ejecutarla

LAZY

≠ VIEW normal

La view re-ejecuta la query en cada lectura · La MV no

`REFRESH MATERIALIZED VIEW`

Ejecuta la query y persiste el resultado físicamente

⚡ ACTION

Principio común: definición declarativa (lazy) + una sola acción dispara toda la cadena de ejecución hacia atrás

- END

